

Session 4 · Deck 4.B3

Fast and Accurate: How Each Improvement Works

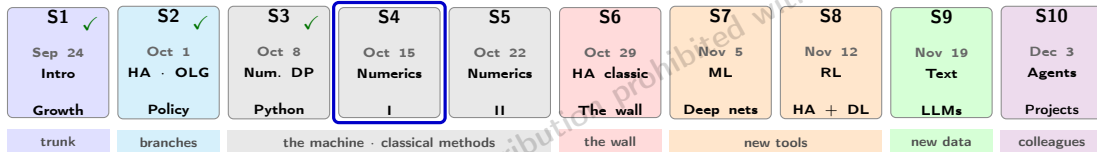
Zhigang Feng

Thursday, October 15, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 4.B3 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 4.B1 approximation and interpolation · 4.B2 expectations: quadrature and the Markov chain · 4.B3 fast and accurate · 4.B4 the worked example, end to end.

Where the Time Actually Goes

The plain solver's bill factorises:

$$\underbrace{n_k n_z n_k}_{\text{one sweep}} \times \underbrace{\frac{\ln \varepsilon}{\ln \beta}}_{\text{patience}} = 1.75\text{M} \times 1352 = \mathbf{2.4 \text{ billion.}}$$

Two independent things to attack:

- **The sweep count**, which belongs to β alone — 1,352 measured against a theoretical 1,375, at $\beta = 0.99$.
- **The cost per sweep**, which is the maximization over n_k candidates at every one of $n_k n_z$ states.

This got worse since Session 3

The same solver needed **228** sweeps in 3.A4 at $\beta = 0.95$. Moving to a quarterly calibration multiplied the patience bill by six, and nothing about the economics changed.

Howard attacks the first. Monotonicity and concavity attack the second. **EGM removes the maximization entirely.** This deck shows *how* each one works; the clock is settled in one table at the end.

Howard, the Intuition: Search Rarely, Commit Often

Every VFI sweep does two jobs. **Search**: find the best k' at each state — n_k comparisons per state, expensive. **Evaluate**: update what each state is worth given those choices — one lookup, cheap. Plain VFI does both, every sweep.

But the search stops finding anything new long before the value converges:

sweep	value error	spread across states	choices not yet final
10	74.8	6.86	3,500
50	47.8	1.04	3,440
100	28.6	0.11	461
232	7.59	0.0003	0
800	0.025	2×10^{-13}	0

From sweep 232 on, the error is the **same number at every state**, and a constant changes no decision:

$$\arg \max_{k'} \{u + \beta \mathbb{E}[v + c]\} = \arg \max_{k'} \{u + \beta \mathbb{E}v\}.$$

Ledger howard_intuition: plain VFI, quarterly RBC, $n_k = 500$, 7-state Tauchen, from $v = 0$; 3,500 states. Session 8 names the two halves *evaluation* and *improvement*.

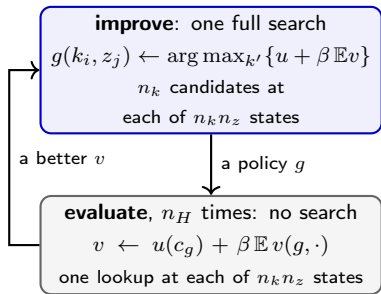
1,120 of 1,352 searches wasted

After sweep 232 the only work left is to raise the *level* of v , and the cheap evaluation step does that just as well. Howard: **search** for a policy, **commit** to it for n_H cheap sweeps to learn what it is worth, then search again.

The trade-off

How long to commit before searching again is a cousin of **exploration vs. exploitation**: exploit the current policy, or look for a better one. Here the search sees every choice exactly, so searching costs only time, never a mistake.

How Howard Works: Evaluate, Then Improve



One costly search buys n_H cheap sweeps. Plain VFI is $n_H = 0$: a search every sweep.

With the policy held fixed there is **nothing left to choose**, so an evaluation sweep is a lookup:

$$v^{(n+1)}(k_i, z_j) = u(c_{ij}) + \beta \sum_m \Pi_{jm} v^{(n)}(g(k_i, z_j), z_m).$$

Why it still converges

Each evaluation sweep shrinks the distance to the value of *that policy* by the factor β — the same contraction as before, minus the max. After n_H sweeps v is nearly that policy's own value; the next search compares choices against a far better v , and a search can only improve the policy.

Let $n_H \rightarrow \infty$ (full policy iteration): every search improves the policy and a grid has finitely many policies, so it stops after finitely many searches — a promise plain VFI cannot make.

Howard, Measured: Searches and Comparisons

	searches	comparisons
plain VFI	1,352	2.37 bn
$n_H = 10$	124	221 m
$n_H = 20$	66	120 m
$n_H = 50$	29	56 m

From 1,352 searches to **29**, at *identical* accuracy — every row has the same worst Euler error, $10^{-1.55}$. The comparisons fall 42-fold; what remains is mostly the cheap evaluation sweeps.

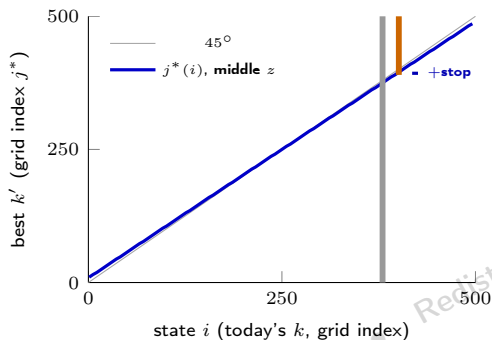
Ledger acceleration: quarterly RBC, $n_k = 500$, 7-state Tauchen; comparisons count the search candidates plus one per state per evaluation sweep. Rule of thumb: $n_H \in [10, 50]$.

Diminishing returns

$n_H : 10 \rightarrow 20$ halves the searches; $20 \rightarrow 50$ halves them again, but each search now carries fifty evaluation sweeps, so the total work falls more slowly.

Accuracy did not move. Howard changes how you reach the fixed point, not which fixed point it is. The discretization error is untouched.

How Monotonicity Works: Search Upward From the Last Answer



The policy index rises with the state. At state 401 grid search tries all 500 candidates (grey); the ratchet starts at the previous answer, $j^* = 390$ (orange); with concavity it also stops just past the peak (blue): the window is only as wide as the policy moved.

Ledger monotone_picture. The ratchet is a sequential loop: in Python it runs slower than one vectorised search; compiled, the saving is real.

$g(k, z)$ is increasing in k : a richer household saves more. So the best k' at state $i + 1$ cannot lie **below** the best k' at state i — skip every candidate below it.

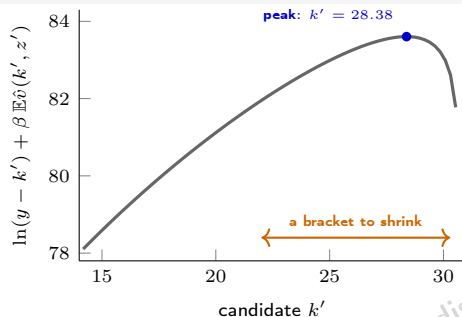
The ratchet

- 1: $j_{\min} \leftarrow 1$
- 2: **for** $i = 1, \dots, n_k$ **do**
- 3: search only $j = j_{\min}, \dots, n_k$; record $j^*(i)$
- 4: $j_{\min} \leftarrow j^*(i)$ ▷ the window only moves up
- 5: **end for**

comparisons per sweep, 3,500 states

grid search: n_k per state	1.75 million
ratchet: the window shrinks as i rises	0.88 million
ratchet + first-decline stop (next frame)	10,400

How Concavity Works: One Peak, So Stop Early — or Shrink a Bracket



The inner objective at one state ($k = 28.38 \approx k_{ss}$, middle z) over every feasible grid k' : **one peak**. It rises, turns once, and plunges as consumption nears zero. Ledger concave_picture.

Why one peak: u is concave in c and $\hat{v}$ in k' (linear interpolation keeps it so, 4.B1), so their sum rises, peaks once, and falls. Two ways to use that:

1. **Stop early.** Walk the candidates upward and stop one past the first decline: nothing beyond can be higher. Alone it saves little here (252 of 291: the peak sits high); with the ratchet starting the walk near the peak, the sweep falls to about $3n_k$ per z .
2. **Shrink a bracket.** Hand $[k'_{\min}, k'_{\max}]$ to 4.A1's golden section: 26 / 36 / 46 evaluations to 10^{-4} / 10^{-6} / 10^{-8} , whatever n_k is — and the answer need not be a grid point.

The second route moves the **accuracy**, not only the cost: the worst Euler error falls from $10^{-1.55}$ to $10^{-2.77}$ because the policy can leave the grid. Combine: ratchet to bracket, golden section inside, Howard around it.

The Endogenous Grid Method: Run the Euler Equation Backwards

Time iteration is expensive because k' appears on *both* sides of

$$u'(y(k, z) - k') = \beta \mathbb{E}[u'(c(k', z'))R(k', z')],$$

so every state needs a root-find, every sweep.

Carroll's insight (2006): treat the grid as *tomorrow's* assets. Fix k' . Then the right-hand side is computable immediately, and the left-hand side inverts:

$$c = (u')^{-1}\left(\beta \mathbb{E}[u'(c(k', z'))R(k', z')]\right).$$

Now you know c and k' — and today's state follows.

The trade

The unknown moves from *inside* a non-linear equation to the *outside* of an explicit formula. For CRRA, $(u')^{-1}(x) = x^{-1/\gamma}$ — one exponentiation.

The grid you end up with is **endogenous**: the k values that generate your chosen k' are wherever the algebra puts them. One interpolation brings the policy back to the grid you wanted.

Carroll, "The method of endogenous gridpoints for solving dynamic stochastic optimization problems", *Economics Letters* (2006).

Getting EGM Right: Put the Grid on Cash on Hand

With the grid on capital, EGM has one root-find left:
given c and k' , today's capital $\tilde{k}$ must solve

$$f(\tilde{k}) + (1 - \delta)\tilde{k} - c - k' = 0,$$

at every node, every sweep — the very cost the method exists to remove.

The fix is to choose the right state. Let

$$m = zk^\alpha + (1 - \delta)k \quad (\text{cash on hand}).$$

Then the budget constraint is simply $m = c + k'$, so

$$\tilde{m} = c + k',$$

with no equation to solve at all.

Why this is the real method

Carroll's formulation is in cash on hand for exactly this reason. Keeping the grid in k brings that root-find straight back.

Measured consequence: our cash-on-hand EGM does **zero** root solves. An implementation that keeps the grid in k needs one per grid point and shock — 3,500 per sweep.

EGM in Four Steps

For each z_j and each end-of-period k' on the exogenous grid:

1. **Next period's cash** for every z' : $m' = z'(k')^\alpha + (1 - \delta)k'$, and $c' = \hat{c}(m', z')$ by interpolation.
2. **The expectation**: $\mathcal{R} = \beta \sum_l \Pi_{jl} u'(c') R(k', z_l)$.
3. **Invert**: $c = (u')^{-1}(\mathcal{R})$. For log utility, $c = 1/\mathcal{R}$.
4. **Today's state**: $\tilde{m} = c + k'$. Interpolate $(\tilde{m}, c)$ back onto the m -grid.

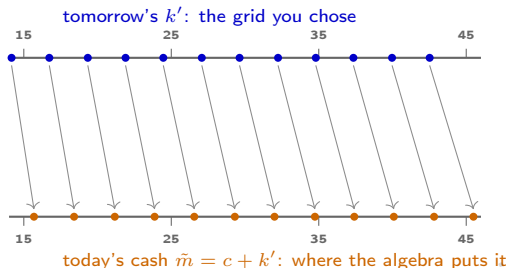
Where m falls below the smallest $\tilde{m}$, the constraint binds and $c = m - \underline{k}$.

Count the solves

Steps 1–4 contain one interpolation, one matrix product, one exponentiation and one addition. **No maximization and no root-finding.**

That is the whole method. It is shorter than the VFI loop it replaces, and it converged here in 420 iterations. The next frame draws one of them.

EGM in a Picture: Tomorrow's Grid, Today's Points



Twelve of the 500 nodes, middle z , one step from the converged policy (ledger `egm_picture`). Each arrow is one Euler inversion; they never cross, so the endogenous points come out **ordered**.

One sweep of the method, from the converged policy:

1. Pick k' **on the grid** (top row).
2. Invert the Euler equation for today's c : at $k' = 27.07$, $c = 2.24$.
3. Add: $\tilde{m} = c + k' = 29.31$ (bottom row). Nothing was solved.

The bottom row is the **endogenous grid**: not where you would have put points, but ordered — so one linear interpolation of the pairs $(\tilde{m}, c)$ returns $c(m)$ on the m -grid you wanted, $[16.06, 45.15]$, 500 points.

Where the constraint binds

Below the first endogenous point, $\tilde{m}_1 = 15.69$, no interior choice exists and $c = m - \underline{k}$. This grid starts at 16.06, so here it never binds; in Aiyagari (S6) it does, by the same rule.

When EGM Does Not Apply

The method needs four things, and each is a real restriction:

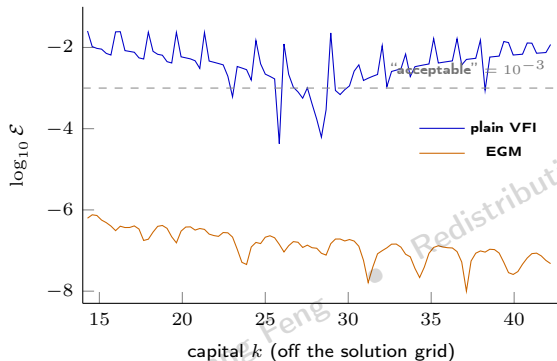
1. u' must be **invertible** in closed form — CRRA, CARA, quadratic. Not satiation.
2. The budget must solve for today's state **explicitly** given (c, k') — which is what cash on hand buys.
3. **One** continuous control. Two, and part of the problem goes back to root-finding.
4. The endogenous grid must come out **ordered**, which needs a monotone policy.

What breaks item 4

Discrete choices — to work or not, to default or not — make the policy non-monotone, the endogenous grid folds over itself, and a plain interpolation picks the wrong branch.

The repair is an **upper-envelope** step: Fella (2014), and Iskhakov, Jørgensen, Rust & Schjerning (2017) for the discrete-continuous case. Both are outside this course, and both are what you would read next.

The Accuracy, Plotted



Unit-free Euler residuals *off* the solution grid.
Plot: 100 capital levels, middle productivity state. Table: 4,000 random (k, z) draws.

	mean	worst
plain VFI	-2.18	-1.55
EGM	-6.75	-6.06

Four and a half orders of magnitude, on the same model and the same chain. VFI's worst case is a 2.8% consumption mistake; EGM's is one part in a million.

Reporting: What Goes in the Paper

1. The method, and the implementation choices that matter.
2. The grid: points, spacing, bounds — and the chain, with its *implied* ρ and σ_z (4.B2).
3. The convergence tolerance.
4. **Maximum and mean Euler errors on an evaluation grid that is not the solution grid.**
5. A grid-density study, and whether the observed rate matches theory.
6. Computation time and hardware.

Item 4 is the one with teeth

Evaluating residuals *on* the solution grid is the numerical analogue of reporting training accuracy. The solution was constructed to satisfy the Euler equation there.

And item 5 has to name which object the rate applies to. 3.A2 measured **1.89 for the value function** and **0.93 for the policy** — quoting a single rate without saying which is how the two get confused.

Den Haan–Marcet is the simulation-based complement, and needs a stochastic model to be non-degenerate.

The Benchmark Table, Recomputed

Method	time	speed-up	worst $\log_{10} \mathcal{E}$	what changed
Plain VFI, grid search	3.50 s	1.0	-1.55	—
+ Howard, $n_H = 20$	0.24 s	14.5	-1.55	fewer maximizations
+ Howard, $n_H = 50$	0.15 s	23.3	-1.55	fewer still
+ monotonicity	0.99 s	3.5	-1.55	fewer comparisons, more overhead
+ continuous choice	0.83 s	4.2	-2.77	the policy leaves the grid
EGM, cash on hand	0.08 s	44.2	-6.06	no max, no root-find

Comparisons of this kind are associated with Aruoba, Fernández-Villaverde & Rubio-Ramírez (2006), which sets VFI, perturbation, Chebyshev projection and finite elements against each other. **The table above is our own run**, and it covers Howard and EGM, which that paper does not.

So this one is ours, on one machine, at one calibration, with the script in the repository. **Cite a benchmark table to the run that produced it.**

Takeaways

- The bill factorises into **sweeps** (which belong to β) and **cost per sweep** (which belongs to the maximization). Different tools attack each.
- **Howard**: search rarely, commit often — one search buys n_H cheap evaluation sweeps, and the policy settles long before the value. 1,352 searches became 29, at identical accuracy.
- **Monotonicity**: a richer state saves more, so start each search where the last one stopped. The ratchet halves the comparisons; with concavity's early stop they fall below one percent.
- **Concavity**: one peak, so a bracket can be shrunk instead of a grid searched — golden section's 26–46 evaluations, and an answer off the grid: $10^{-1.55} \rightarrow 10^{-2.77}$.
- **EGM**: fix *tomorrow's* grid, invert u' for c , and today's state is $\tilde{m} = c + k'$ with nothing to solve — but only with the grid on **cash on hand**. Fastest and most accurate: $10^{-6.06}$.
- Report residuals **off** the solution grid, and cite a benchmark table to the run that produced it.

Bridge: What You Owe, and What Comes Next

Session 4 in one sentence. Every crude choice of Session 3 had a repair; we built each one and measured what it bought — and the largest single gain came from *reformulating the state*, not from a better algorithm.

- **4.B4** puts the whole toolkit into one program: the stochastic model solved again, line by line, against the crude solver.
- **Lab L04** runs all of it: the root-finder bake-off, golden section, the interpolation shape tests, quadrature, Tauchen against Rouwenhorst, and the accelerated RBC.
- **Homework 1, Part B** is out today; both parts are due October 29.

Next week (S5)

Numerical Methods II. Everything so far has put values on a grid. **Perturbation** throws the grid away and expands around the steady state; **projection** replaces N separate values with a handful of basis coefficients.

Both are then run on *this* model, side by side. Aiyagari follows in Session 6.

And two threads from 4.A2 are left open on purpose: **CG and SGD** return in **S7**, as the optimizers that actually train a network, and the **Metropolis–Hastings** rule returns in **S9**, where the object being sampled is a posterior rather than a maximum.